

MANUAL26. УРОК 1. ВВЕДЕНИЕ В ТЕСТИРОВАНИЕ: ОШИБКА, ДЕФЕКТ, СБОЙ. РОЛЬ QA.

Что такое тестирование

Тестирование

это проверка того, как продукт ведет себя в реальных условиях. Не в идеальном сценарии, а в ситуации, когда пользователь может ошибиться, прервать действие, работать с другой версией приложения или слабым интернетом.

Смысл тестирования не в «поиске проблем ради отчета», а в том, чтобы команда понимала фактическое состояние продукта.

У тестировщика обычно три опорных вопроса: что

ожидалось, что получилось на деле и какой риск из этого следует. Если вы держите эту логику в голове, то даже сложные кейсы становятся разборчивыми.

Почему важно изучать тестирование с самого начала

Многие представляют QA как роль, которая появляется в конце разработки. На практике QA подключается намного раньше: когда уточняются требования, обсуждаются критерии приемки и выбираются рискованные сценарии. Чем раньше замечена проблема, тем дешевле и спокойнее ее исправление.

Этот материал дает базу, без которой дальше будет тяжело: вы разберетесь в связке error -> defect -> failure, поймете, как QA участвует в жизненном цикле продукта и научитесь записывать наблюдения так, чтобы по ним можно было воспроизвести проблему.

В продуктовой разработке тестирование нужно для управляемости, а не для формальности. Команде важно видеть, что уже проверено, где

остаются риски и какие изменения могут поломать ключевой пользовательский путь. Когда такой картины нет, релизы выходят «на ощущениях».

Перед следующим абзацем зафиксируем ключевые термины. Error — это ошибка человека в решении. Defect — проблема, которая закрепилась внутри системы. Failure — внешнее проявление этой проблемы, которое уже заметно пользователю.

Базовая модель этой темы — error -> defect -> failure. Error — это человеческая ошибка в решении: на этапе требований, дизайна или реализации. Defect — это то, во что эта ошибка превратилась внутри системы: неверная логика, условие, данные, конфигурация. Failure — внешнее проявление дефекта, которое уже заметно пользователю.

Разберем пример, когда в требованиях не уточнили правило валидации (error). В коде сделали слишком мягкую проверку (defect). Пользователь сохраняет данные, но позже получает ошибку в отчете (failure). Эта цепочка полезна тем, что разделяет причину и

СИМПТОМ: ВЫ ВИДИТЕ, ЧТО ИМЕННО
исправлять и где.

Термин QA (Quality Assurance) в этом контексте означает системную работу с качеством, а не только проверку готовой сборки.

Роль QA в этой логике намного шире, чем запуск набора тестов. QA помогает уточнять требования, превращать расплывчатые формулировки в проверяемые условия, заранее выделять рискованные сценарии и фиксировать результат в понятном формате. После исправлений QA проверяет, не задел ли фикс соседние части системы, то есть контролирует регрессию.

SDLC (Software Development Life Cycle)

это жизненный цикл разработки продукта: от идеи и требований до релиза и поддержки.

Если смотреть на SDLC целиком, QA нужен на каждом этапе: от обсуждения требований до анализа инцидентов после релиза. На ранних этапах QA снижает вероятность дорогих

переделок, на поздних — снижает вероятность пользовательских проблем в продакшене.

Отдельный практический навык — формулировка дефекта. Фраза «не работает» для команды почти бесполезна. Рабочая запись должна включать контекст, шаги, ожидаемый результат и фактический результат. Тогда дефект можно быстро воспроизвести, обсудить и правильно приоритизировать.

На уровне первого урока достаточно понимать три типа проверок. Функциональные отвечают за то, делает ли система то, что обещано. Нефункциональные — насколько стабильно, быстро и удобно это работает. Регрессионные — не сломали ли новые изменения то, что раньше работало.

Коммуникация QA всегда опирается на факты и аккуратный язык. Важно отделять наблюдение от гипотезы и показывать риск для пользователя. Чем яснее формулировка, тем быстрее команда принимает решение.

Давайте разберем на кейсе по игре GTA5

Пример 1. Не засчитывается прохождение миссии

В этой ситуации в логике проверки этапа не учтен один из допустимых способов выполнения (error), из-за чего условие завершения миссии реализовано неверно (defect), и игрок выполняет задачу, но миссия не засчитывается (failure).

Пример 2. Вылет во время полицейской погони

Здесь все начинается с недооценки нагрузки на клиент в насыщенной городской сцене (error), затем проявляется как нестабильная обработка ресурсов (defect), а пользователь видит вылет во время погони (failure).

Пример 3. Некорректный маршрут на GPS

В этом кейсе неверно настроено обновление цели на карте (error), поэтому маршрут ведет в неправильную точку (defect), и игрок едет не туда, теряя время (failure).

Далее представлена таблица, которая сводит данные для

удобного отображения:

Ситуация/сценарий	Возможная ошибка	Дефект/баг в системе	Failure для пользователя	Риск
Миссия не засчитывается	Не учтен один из возможных сценариев прохождения	Условие завершения миссии задано неверно	Игрок выполняет этап, но миссия не закрывается	Потеря прогресса и раздражение
Вылет во время погони	Не верно оценена нагрузка городской сцены	Клиент нестабильно работает под нагрузкой	Игра закрывается в середине погони	Потеря времени и сессии
GPS ведет в неверную точку	Неверно настроен сценарий обновления	Маршрут на карте обновляется ошибочно	Игрок едет не туда и срывает темп миссии	Потеря прогресса и раздражение

Типичные ошибки новичков и корректирующие формулировки

Частая ошибка — смешивать error, defect и failure в одно понятие «баг». Если пользователь видит проблему на экране, это еще не error: сначала нужно отделить внешний симптом от причины в системе.

Вторая ошибка — писать без условий. Формулировка «миссия сломана» не помогает, если не указаны платформа, версия и шаги. Третья ошибка — эмоциональная оценка вместо факта. Суждение «игра ужасная» не решает задачу, а точное описание ожидаемого и фактического результата решает.

Еще один важный момент — всегда показывать риск. Дефект

без объяснения последствий
обычно получает низкий
приоритет. Дефект с понятным
пользовательским ущербом
обсуждается быстрее и по делу.

Итоги занятия

Если вы можете описать
проблему без общих слов,
разделить error/defect/failure и
объяснить риск для
пользователя, то фундамент
первого урока у вас есть.

Критерии успеха:

- Вы без подсказки объясняете разницу между error, defect и failure.
 - В ответах есть условия и наблюдаемые факты, а не общие жалобы.
 - Вы понимаете, что QA — это не только финальная проверка перед релизом.
 - В разборе кейса вы можете назвать пользовательский риск.
-

Глоссарий к занятию

1. Testing (тестирование) —
Процесс оценки качества ПО
через проверку ожидаемого и
фактического поведения.

2. Quality Assurance (QA) —
Подход и практики, которые
помогают обеспечивать
качество продукта на всем
жизненном цикле.
3. Requirement (требование) —
Ожидание к системе: что она
должна делать и при каких
условиях.
4. Error (ошибка) — Действие
человека, которое приводит к
неверному решению в
требованиях, дизайне, коде
или данных.
5. Defect / Bug (дефект) —
Проблема в артефакте (коде,
требованиях, конфигурации,
данных), которая может
вызвать неправильное
поведение.
6. Failure (сбой) — Наблюдаемое
отклонение поведения
системы от ожидаемого
результата во время
выполнения.
7. Test Case (тест-кейс) —
Структурированная проверка:
условия, шаги, ожидаемый
результат, фактический
результат.
8. Risk (риск) — Вероятность
нежелательного события и
масштаб его последствий для
пользователя и бизнеса.